

FPGA Tick-to-Trade

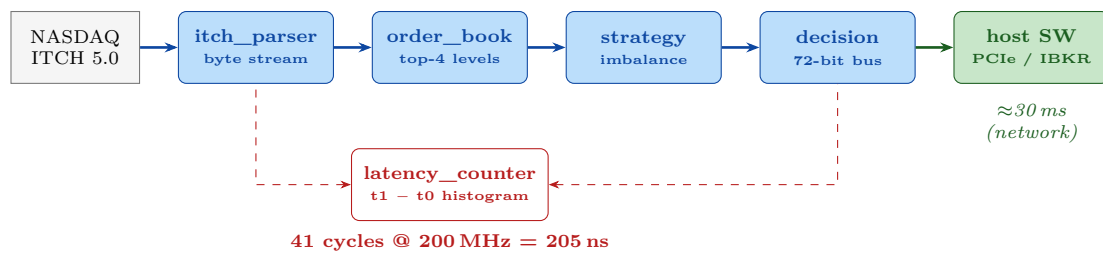
Technical Brief • June 2026

A hardware trading pipeline: NASDAQ ITCH in, order decision out, in **205 ns**.

This system reconstructs the NASDAQ order book and emits a trading decision **entirely in FPGA fabric**, on a fixed, jitter-free 205 ns budget — roughly **five orders of magnitude** faster than the equivalent software path. It targets AWS F1 (Xilinx UltraScale+ VU9P), is verified bit-for-bit against a Python golden model, and routes its decisions to a live Interactive Brokers *paper* account. This brief is the map; the three detailed parts (architecture, RTL & verification, integration) are the territory.

1. System architecture

The design has two paths. The **hardware fast path** (blue) turns a raw market-data byte stream into a trade decision in 205 ns. The **software execution path** (green) takes that decision off-chip and routes it to the broker — network-bound at tens of milliseconds. The whole point of the project lives in the gap between the two.



- **One clock domain, one direction.** A byte enters every cycle; a decision leaves a fixed 41 cycles after a message completes. No floating-point anywhere in the data path — all fixed-point integer arithmetic.
- **Valid/ready back-pressure** between the parser and the book means a message arriving while the book is busy is *held*, never dropped.
- **An on-chip cycle counter** stamps the entry and exit of every message, so latency is *measured*, not estimated — and read back over AXI-Lite.

2. Pipeline stages

Module	Role	Key property
itch_parser	AXI-Stream byte decoder: turns a framed ITCH message into structured fields (type, timestamp, refs, side, shares, price).	1 byte/cycle, held output
order_book_m2	Direct-mapped 256-entry book; maintains the top 4 price levels per side, aggregating size at each level.	O(1) Add, RESCAN on edits
strategy_imbalance	Depth-weighted book-imbalance signal with spread guard, cooldown and imbalance-scaled lot sizing.	1 cycle, no division
latency_counter	Free-running cycle counter + 64-bucket histogram of tick-to-trade latency, read over AXI-Lite.	single-cycle precision
tick_to_trade_top	Top-level wrapper: wires the four stages and packs the decision into a 72-bit bus.	72-bit decision out

Decision bus (72 bits): [71:65] reserved • [64] action (0=BUY, 1=SELL) • [63:32] limit price (fixed-point $\times 10,000$) • [31:0] size (shares). Read by the host over PCIe DMA, decoded, and risk-checked before it reaches the broker.

3. Key design decisions

1. Determinism over throughput tricks

The order book keeps top-of-book correct in **O(1)** on every Add. It only pays for a full re-scan when an order at a *displayed price level* is cancelled, deleted or executed — a cancel deep in the book is skipped entirely.

Combined with the back-pressure handshake, the result is a pipeline whose latency is a **single fixed number**, not a distribution. A CPU, with its caches, interrupts and OS scheduler, cannot offer this; an execution desk prices determinism directly.

2. Fixed-point cross-multiply — no dividers, no floats

The imbalance test $\text{bid}/(\text{bid}+\text{ask})$ is never computed as a division. It is rearranged into a cross-multiply, e.g. BUY when $\text{bid_size} \times 10 > \text{BID_THRESH} \times \text{ask_size}$. One DSP multiply per side, one clock cycle, exact integer arithmetic — no FPU, no multi-cycle divider, no rounding error.

3. The order book is RAM, not flip-flops

The $256\text{-entry} \times 130\text{-bit}$ order table is a memory, not 33,280 registers. The per-slot “occupied” flag was pulled into a small separate bit-vector so the payload infers distributed RAM with combinational (zero-cycle) reads — keeping the book $O(1)$ and synthesizable without exploding the flip-flop count.

4. Correct capture at the message boundary (the NBA fix)

When a field’s last byte coincides with AXI-Stream `tlast`, a naive non-blocking assignment captures the *pre-shift* value and drops that byte. The parser computes combinational “next” values that include the current byte, so the output latched at `tlast` is always complete. This single subtlety was the root cause of the first generation of failing tests.

4. Market-data coverage

Eight ITCH 5.0 message types are decoded. Replace carries *two* order references and shifted field offsets; Trade is informational (the book treats it as a no-op); Add-with-MPID and Execute-with-Price reuse Add/Execute field positions.

Type	Message	Bytes	Effect on book
A	Add Order	36	insert order, update level
F	Add Order with MPID	40	insert (same as A; MPID ignored)
X	Order Cancel	23	reduce shares
D	Order Delete	19	remove order
E	Order Executed	31	reduce shares
C	Order Executed with Price	36	reduce shares (exec price logged)
U	Order Replace	35	remove original, insert replacement
P	Trade (non-cross)	44	none (informational print)

5. Strategy and risk controls

The signal is short-horizon **book imbalance**, a well-known micro-structure predictor: a bid much heavier than the ask anticipates an up-move (BUY), and vice-versa. Three controls turn the raw signal into something an actual desk would run:

- **Depth-weighted volume.** Imbalance is computed over the top 4 levels, weighting the touch most heavily (weight 4) down to the deepest tracked level (weight 1) — so a *supported* bid fires even when the touch alone looks balanced.
- **Spread guard.** No trade unless the book is well-formed ($\text{ask} > \text{bid}$) and the spread is within `MAX_SPREAD_TICKS` (\$0.10) — avoids chasing fills in thin or locked markets.
- **Cooldown + scaled sizing.** After firing, new decisions are suppressed for `COOLDOWN_CYCLES` (no order spam); order size scales with imbalance strength from `BASE_LOT=100` up to `MAX_LOT=250` shares.

Off-chip, `risk_guard.py` applies pre-trade checks (SEC Rule 15c3-5) before any order is sent, and the live path only ever targets a paper account.

6. Latency measurement (how the 205 ns is proven)

A free-running 32-bit counter timestamps `t0` on the first byte of a message and `t1` on `decision_valid`; $\Delta = t1 - t0$ is binned into a 64-bucket histogram. Because the pipeline is deterministic, that histogram is a **single spike**

at 41 cycles — visual proof of zero jitter, where a CPU would smear across 3–4 orders of magnitude. The host reads it over a small AXI-Lite slave:

Address	Register	Access
0x000 – 0x0FC	histogram buckets 0–63	read
0x100	last measured latency (cycles)	read
0x104	clear (write 1 to reset)	write

7. Verification

Every RTL output is diffed **bit-for-bit** against an independent Python golden model (`golden/itch_parser.py`) under cocotb + QuestaSim. The same generated vectors drive both implementations; any field mismatch fails the test. All RTL also passes a strict `verilator -Wall` lint.

Testbench	Tests	Covers
tb_itch_parser	11	all 8 message types, back-to-back, back-pressure hold
tb_order_book	11	M1 top-of-book on Add (baseline)
tb_order_book_m2	28	multi-level depth, RESCAN, replace, RESCAN-skip, no-op trade
tb_strategy_imbalance	21	imbalance, spread guard, cooldown, lot tiers, depth weighting
tb_latency_counter	5	histogram, saturation, AXI-Lite read, clear
tb_tick_to_trade	5	full pipeline, 205 ns latency, back-pressure no-drop
Total	81	100 % pass

8. Hard numbers

Metric	This design (FPGA)	Software pa
Tick-to-trade <i>compute</i>	205 ns (41 cycles)	~1 μ s (C, -O2); ~60 μ s (Python)
Compute speed-up (measured)	~5 $\times$ vs compiled C – ~300 $\times$ vs Python (same golden chain)	
Latency jitter	0 ns (deterministic)	~0.4–1 ms tail spikes (OS-scheduled, unbounded)
Worst-case (tail) vs FPGA	C tail reaches ~1,800 $\times$ the FPGA – the tail, not the median, is what HFT loses on	
+ Broker network round-trip	— (separate)	~30 ms (additional, on the live path)
Clock frequency	200 MHz (5 ns period)	
ITCH ingest rate	1 byte/cycle = 200 MB/s	
Order book	top-4 levels/side, 256 live orders	
ITCH message types decoded	8 (A, F, C, D, E, U, X, P)	
Data-path arithmetic	fixed-point, 0 floats	
Verification	81/81 tests, golden-model diff	

9. Status and honest claims

Done: all RTL written, lint-clean, 81/81 cocotb tests passing; latency measured at 205 ns by the on-chip counter *in cycle-accurate simulation*; host software (bridge, risk guard, golden model, live feed) in place.

Pending: Vivado synthesis / timing closure on the VU9P part (to be run on a larger-RAM machine), PCIe DMA read path, and end-to-end live paper-trading runs. The 205 ns figure is the design target validated in simulation by the hardware counter; it has not yet been re-confirmed on physical silicon.

Toolchain: SystemVerilog RTL • cocotb 2.x + QuestaSim 2021.2 • Verilator lint • Vivado ML (synthesis) • Python golden model + IBKR `ib_async`. **Target:** AWS F1 f1.2xlarge, Xilinx UltraScale+ VU9P.